



Strings in C

Understanding Character Arrays and String Operations



Declaration



Initialization



Functions





Declaration and Initialization of Strings



Strings are actually **one-dimensional array of characters** terminated by a null character `'\0'`.

Key Points:

- ✓ A null-terminated string contains characters that comprise the string followed by a null character.
- ✓ The size of character array containing a string is one more than the number of characters in the word.

```
char greeting[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
```



Example: The above declaration creates a string consisting of the word "Hello" with 6 characters plus null terminator.





Memory Layout of Strings



Understanding how strings are stored in memory is crucial for proper string manipulation.

Memory Representation:

```
1 byte 1 byte 1 byte 1 byte 1 byte 1 byte
P R O G R A M \0
```

Each character of string occupies 1 byte of memory (on 16-bit computing).



Contiguous Memory

The characters of strings are stored in contiguous (adjacent) memory locations.



Machine Dependency

The size of character is machine dependent, and varies from 16-bit to 64-bit computers.



String Constants

 Strings in C are a group of characters, digits, and symbols enclosed in quotation marks.

Key Characteristics:

- ✓ Can be assigned to char pointers or char arrays
- ✓ The end of the string is marked with a special character '\0'

```
char greeting[] = "Hello";  
char greeting[] = "Hello";
```



Key Difference: The first statement is not a string but simply an array storing three characters 'H', 'e', 'l', 'l', 'o', whereas the second one is a four character string "Hello\0".





String Examples



To declare a string, specify the data type as char and place the number of characters in array in square brackets after the string name.

Syntax:

```
char string-name[size];
```



Example Declarations:

```
char name[20];  
char address[25];  
char city[15];
```



Initialization:

The string can be initialized with or without specifying size.



Built-in String Functions

 The header file [string.h](#) contains some string manipulation functions.

Common Functions:

- ✓ `strlen()` - Returns string length
- ✓ `strcpy()` - Copies one string to another
- ✓ `strcmp()` - Compares two strings
- ✓ `strcat()` - Joins one string to another
- ✓ `strlwr()` - Converts to lowercase
- ✓ `strrev()` - Reverses a string
- ✓ `strspn()` - Finds string position



Note: These functions operate on null-terminated arrays of characters and require the header .





Strlen Function



The **strlen function** returns the length of a string. It takes the string name as argument.

Syntax:

```
n = strlen(str);
```

where str is name of the string and n is the length of the string, returned by strlen function.

```
#include  
#include  
  
int main() {  
    char greeting[6] = {'H', 'e', 'l', 'l', 'o', '\0'};  
    printf("Greeting message: %s\n", greeting);  
    return 0;  
}
```

Output:

```
Greeting message: Hello
```





Key Point: The `strlen` function counts characters until it encounters the null terminator `'\0'`, not including it in the length count.



Strcpy Function



In C, you **cannot simply assign** one character array to another. You have to copy element by element.

Solution:

The string library contains a function called **strcpy** for this purpose.

```
strcpy(str1, str2);
```

where `str1` and `str2` are two strings. The content of string `str2` is copied on to string `str1`.





Important: Strcpy automatically copies the null terminator, so the destination string must have enough space to accommodate it.



Strcmp Function



The **strcmp function** in the string library function which compares two strings, character by character and stops comparison when there is a difference in ASCII value or end of any one string.

Return Values:

- ✓ Zero means the two strings are equal
- ✓ Negative value means first is less than second
- ✓ Positive value means first is greater than second

```
n = strcmp(str1, str2);
```

where str1 and str2 are two strings to be compared and n is the returned value of differed characters.





Use Case: Often used with if statements to determine program flow based on string comparison.



Other String Functions

Strcat Function

Used to join one string to another.

```
strcat(str1, str2);
```

Characters of the second string will be appended to the first string.

Strlwr Function

Converts upper case characters of string to lower case.

```
strlwr(str1);
```

Strrev Function

Reverses the given string.

```
strrev(str1);
```





Advanced Functions: strncpy, strcmp, strncmp, strchr, and others provide more specialized string manipulation capabilities.



Complete Example



Program that reads a name from keyboard and displays message Hello along with your name.

```
#include
#include

int main() {
    char name[10];

    printf("\nEnter Your Name: ");
    scanf("%s", name);

    printf("Hello %s\n", name);
    return 0;
}
```

Output:



```
Enter Your Name: Raghav
Hello Raghav
```



Key Points: The scanf function with %s format automatically reads characters until whitespace is encountered and adds null terminator.



Conclusion

abc

Arrays

All elements are stored in consecutive memory locations



Indexing

Arrays are indexed from 0 up to bound-1



Initialization

Can be done at declaration with braces



Mastering strings is essential for effective C programming!

